A report on

"OOP's Using JAVA"

Submitted in partial fulfilment of the requirements for the award of degree

BACHELOR OF ENGINEERING

IN

Computer Science and Engineering with PwC

By

VISHAD JAIN (25BTRPW104)

Under the guidance of

B **LAVANYA**

Assistant Professor

Department of CSE - Internet of Things

2025-2026

C E R T I F I C A T E

This is to certify that the report entitled "***OOP's Using JAVA***" is a Bonafide work carried out by ***VISHAD JAIN (25BTRPW104)*** in partial fulfilment for the award of degree of Bachelor of Technology in "Computer Science and Engineering with PwC" from Jain - deemed to be University, Bangalore during the year 2025-2026. The report has been approved as it satisfies the academic requirements in respect of laboratories prescribed for the Bachelor of Technology Degree.

Signature of the Faculty

B Lavanya

Assistant professor

Department - CSE- Internet of Things

INDEX

Sl. No	Name	Pg. No	Signature
1	Loan Eligibility Program using If–Else	4	
2	Scholarship Eligibility using Nested If–Else	5	
3	Display Roll Numbers using For Loop	6	
4	Student Grade Evaluation using If–Else, Switch and Loop	7-8	
5	Student Class to Store and Display Details	9	
6	Circle Class using Constructor Overloading	10	
7	Method Overloading for Area Calculation	11	
8	Use of Static and Final Keywords in Java	12	
9	Use of this Keyword to Resolve Ambiguity	13	
10	Inheritance in Robotics System (Single & Multilevel)	14	
11	Employee Management System using Inheritance	15	
12	Runtime Polymorphism using Method Overriding	16	

1. Loan Eligibility Program

Question: A bank wants to develop a simple program to check whether a customer is eligible for a loan. If salary ≥ ₹30,000 → Eligible, otherwise not eligible.

Code:

```
import java.util.Scanner;

public class main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your Salary: ");
        double salary = scanner.nextDouble();

        if(salary >= 30000){
            System.out.println("You are elgible for Loan!");
        }
        else{
            System.out.println("Your are not eligibile!");
        }
        scanner.close();
    }
}
```

Output:

```
PS E:\Inferno\STUDY\College\SEM-2\Java\Class assignment> cd "e:\
; if ($?) { java main }
Enter your Salary: 30000
You are elgible for Loan!
PS E:\Inferno\STUDY\College\SEM-2\Java\Class assignment> cd "e:\
; if ($?) { java main }
Enter your Salary: 29999
Your are not eligibile!
```

2. Scholarship Eligibility Program

Question: Determine scholarship based on marks and family income.

Code:

```
import java.util.Scanner;

public class main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the Marks: ");
        double marks = scanner.nextDouble();
        System.out.print("Enter your Family income: ");
        double income = scanner.nextDouble();

        if (marks >= 75) {
            if (income <= 200000) {
                System.out.println("FULL SCHOLARSHIP GRANTED!");
            } else if (income <= 500000) {
                System.out.println("PARTIAL SCHOLARSHIP GRANTED!!!");
            } else {
                System.out.println("Not elgibile for scholarship!");
            }
        } else {
            System.out.println("Your marks are not eligibile for scholarship!");
        }
        scanner.close();
    }
}
```

Output:

```
PS E:\Inferno\STUDY\College\SEM-2\Java\Class assignment>
; if ($?) { java main }
Enter the Marks: 75
Enter your Family income: 150000
FULL SCHOLARSHIP GRANTED!
```

3. Roll Number Display

Question: Print roll numbers from 1 to 10.

Code:

```
public class main {
    public static void main(String[] args) {
        for (int i = 1; i < 11; i++) {
            System.out.println("Roll number: " + i);
        }
    }
}
```

Output:

```
Roll number: 1
Roll number: 2
Roll number: 3
Roll number: 4
Roll number: 5
Roll number: 6
Roll number: 7
Roll number: 8
Roll number: 9
Roll number: 10
```

4. Student Grade Evaluation

Question: Assign grades and use switch + loop.

Code:

```
import java.util.Scanner;

public class main {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      char choice;
      char grade;
      int marks;

      do {
         System.out.print("Enter the marks: ");
         marks = scanner.nextInt();

         if (marks >= 90) {
            grade = 'A';
         } else if (marks >= 75) {
            grade = 'B';
         } else if (marks >= 50) {
            grade = 'C';
         } else {
            grade = 'F';
         }

         switch (grade) {
            case 'A' -> System.out.println("Excellent");
            case 'B' -> System.out.println("Good");
            case 'C' -> System.out.println("Average");
            default -> System.out.println("Fail!");
         }

         System.out.print("Continue? (Y/N): ");
         choice = scanner.next().charAt(0);
      } while (choice == 'Y' || choice == 'y');
      scanner.close();
   }
}
```

Output:

```
PS E:\Inferno\STUDY\Coll
; if ($?) { java main }
Enter the marks: 80
Good
Continue? (Y/N): Y
Enter the marks: 50
Average
Continue? (Y/N): n
```

5. Student Class Program

Question: A college needs a program to store and display student details such as name, roll number, and marks.

Code:

```
 class Student {
   String name;
   int rollNo;
   int marks;

   Student(String name, int rollNo, int marks ){
      this.name = name;
      this.rollNo = rollNo;
      this.marks = marks;
   }

public static void main(String[] args) {
      Student s1 = new Student("Vishad", 104, 95);
      Student s2 = new Student("Mihir", 61, 95);

      System.out.println("Student #1 Name: " + s1.name);
      System.out.println("Student #1 Roll Number: " + s1.rollNo);
      System.out.println("Student #2 Name: " + s2.name);
      System.out.println("Student #2 Roll Number: " + s2.rollNo);
   }
}
```

Output:

```
Student #1 Name: Vishad
Student #1 Roll Number: 104
Student #2 Name: Mihir
Student #2 Roll Number: 61
```

6. Define a class Circle with Overloaded Constructors

Question: A graphics application needs to create circles in two ways:

1. With a **default radius**
2. With a **user-defined radius**

To support both cases, constructor overloading is required.

Code:

```
public class Circle {
   double radius;

   Circle() {
      radius = 1.0;
   }
   Circle(double radius) {
      this.radius = radius;
   }
   void displayArea() {
      double area = 3.14 * radius * radius;
      System.out.println("Area: " + area);
   }
   public static void main(String[] args) {
      Circle circle1 = new Circle();
      Circle circle2 = new Circle(5.0);
      System.out.println("Circle 1:");
      circle1.displayArea();
      System.out.println("Circle 2:");
      circle2.displayArea();
   }
}
```

Output:

```
Circle 1:
Area: 3.14
Circle 2:
Area: 78.5
```

7. Method Overloading

Question: A math utility program must calculate the area of different shapes

Code:

```
public class main {
    static void calculateArea(double side) {
        System.out.println("Square area: " + (side * side));
    }

    static void calculateArea(double l, double b) {
        System.out.println("Rectangle Area: " + (l * b));
    }

    public static void main(String[] args) {
        calculateArea(5);
        calculateArea(5, 2);
    }
}
```

Output:

```
Square area: 25.0
Rectangle Area: 10.0
```

8. Static and Final

Question: A college wants to store the college name (constant) and count how many students are created.

Code:

```
public class Student {
    final static String College_Name = "Jain University";
    static int count = 0;
    String name;

    Student(String name) {
        this.name = name;
        count++;
    }

    public static void main(String[] args) {
        Student s1 = new Student("Vishad");
        Student s2 = new Student("Mihir");

        System.out.println("Total count of " + Student.College_Name + "'s Student: " +
Student.count);
    }
}
```

Output:

```
Total count of Jain University's Student: 2
```

9. this Keyword

Question: A pet management system stores dog details.

Code:

```
public class Dog {
    String name;
    String breed;

    Dog(String name, String breed) {
        this.name = name;
        this.breed = breed;
    }

    void output(){
        System.out.println("Name of the Dog: " + name);
        System.out.println("Breed of the Dog: " + breed);
    }

    public static void main(String[] args) {
        Dog d1 = new Dog("King" , "labrador");
        Dog d2 = new Dog("Shiro" , "German Shephard");

        d1.output();
        d2.output();

    }
}
```

Output:

```
Name of the Dog: King
Breed of the Dog: labrador
Name of the Dog: Shiro
Breed of the Dog: German Shephard
```

10. Inheritance (Robot)

Code:

```
class Robot {
    void move() {
        System.out.println("Robot is moving");
    }
}

class ServiceRobot extends Robot {
    void assist() {
        System.out.println("Service Robot is assisting humans");
    }
}

class AdvancedRobot extends ServiceRobot {
    void aiTask() {
        System.out.println("Advanced Robot is performing AI task");
    }

    public static void main(String[] args) {
        AdvancedRobot r = new AdvancedRobot();
        r.move();
        r.assist();
        r.aiTask();
    }
}
```

Output:

```
Robot is moving
Service Robot is assisting humans
Advanced Robot is performing AI task
```

11. Employee Management

Code:

```
class Employee {
   String name; int salary;
}
class Manager extends Employee {
   String department;
}
class SeniorManager extends Manager {
   int bonus;
   void display() {
      System.out.println("Name: " + name);
      System.out.println("Salary: " + salary);
      System.out.println("Department: " + department);
      System.out.println("Bonus: " + bonus);
   }
   public static void main(String[] args) {
      SeniorManager s = new SeniorManager();
      s.name = "Vishad";
      s.salary = 75000;
      s.department = "Finance";
      s.bonus = 20000;
      s.display();
   }
}
```

Output:

```
Name: Vishad
Salary: 75000
Department: Finance
Bonus: 20000
```

12. Runtime Polymorphism

Code:

```
 class Employee {
   void calculateSalary() {
      System.out.println("Base Salary");
   }
}
class FullTimeEmployee extends Employee {
   void calculateSalary() {
      System.out.println("Full Time Employee");
      System.out.println("Fixed Monthly Salary: 80000");
   }
}
class PartTimeEmployee extends Employee {
   void calculateSalary() {
      System.out.println("Part Time Employee");
      System.out.println("Hourly Salary: 300/hr");
   }
   public static void main(String[] args) {
      Employee employees;
      employees = new FullTimeEmployee();
      employees.calculateSalary();
      System.out.println();
      employees = new PartTimeEmployee();
      employees.calculateSalary();
   }
}
```

Output:

```
Full Time Employee
Fixed Monthly Salary: 80000

Part Time Employee
Hourly Salary: 300/hr
```